

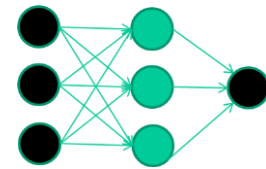
A Brief Overview of Deep Learning

What is Deep Learning?

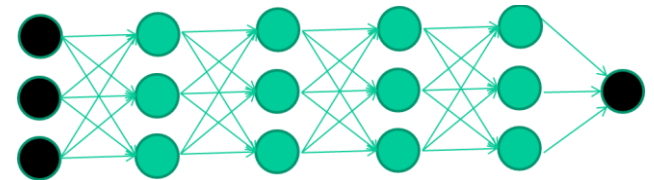
- **‘Deep Learning’ means using a neural Network with several layers of nodes** between input and output
- **The series of layers between input & output do feature identification and processing in a series of stages, just as our brains seem to.**

Multilayer Neural Networks have been around
for **25 years**. **What's actually new?**

we have always had good algorithms for learning the weights in networks with 1 hidden layer

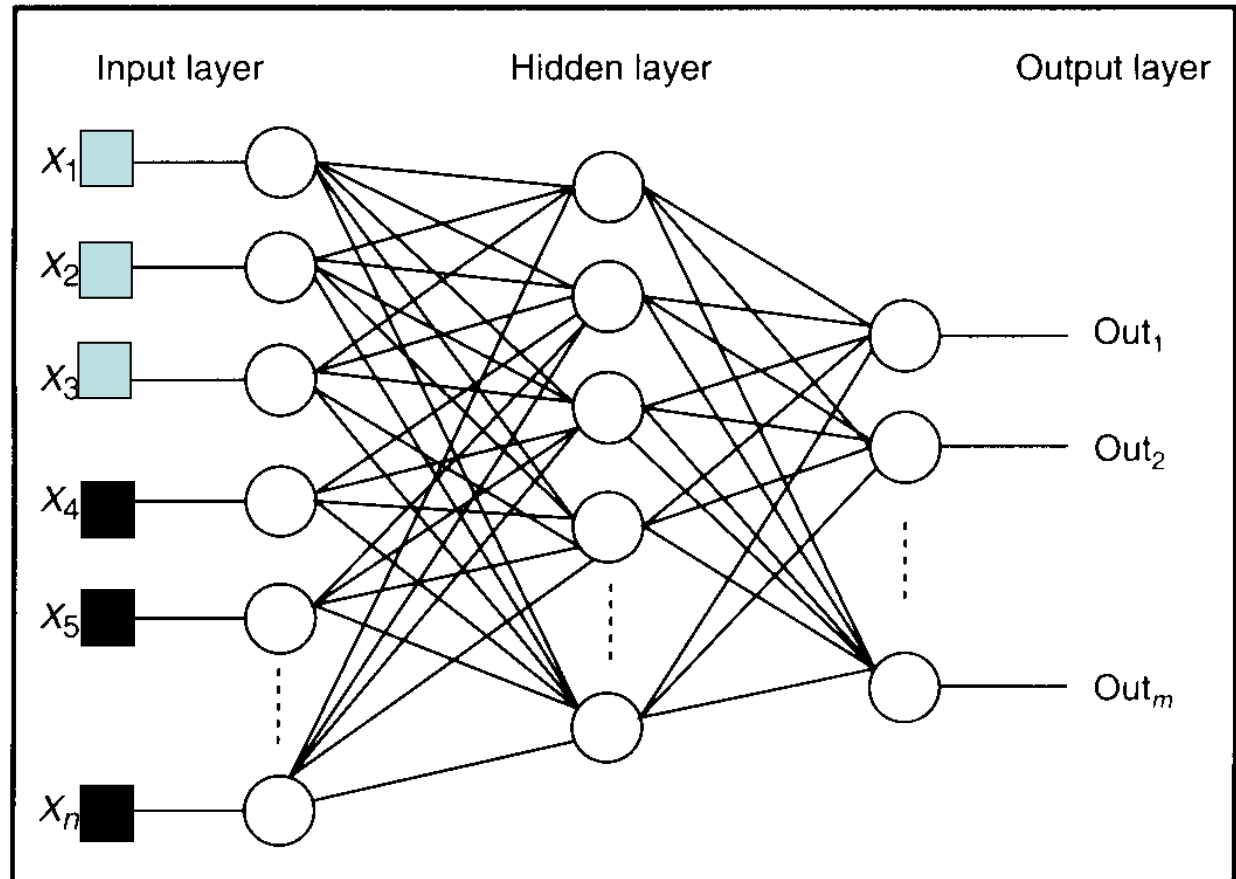
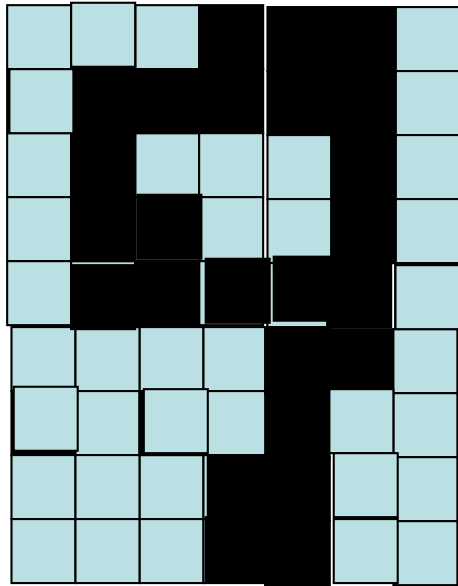


but these algorithms are not good at learning the weights for networks with more hidden layers

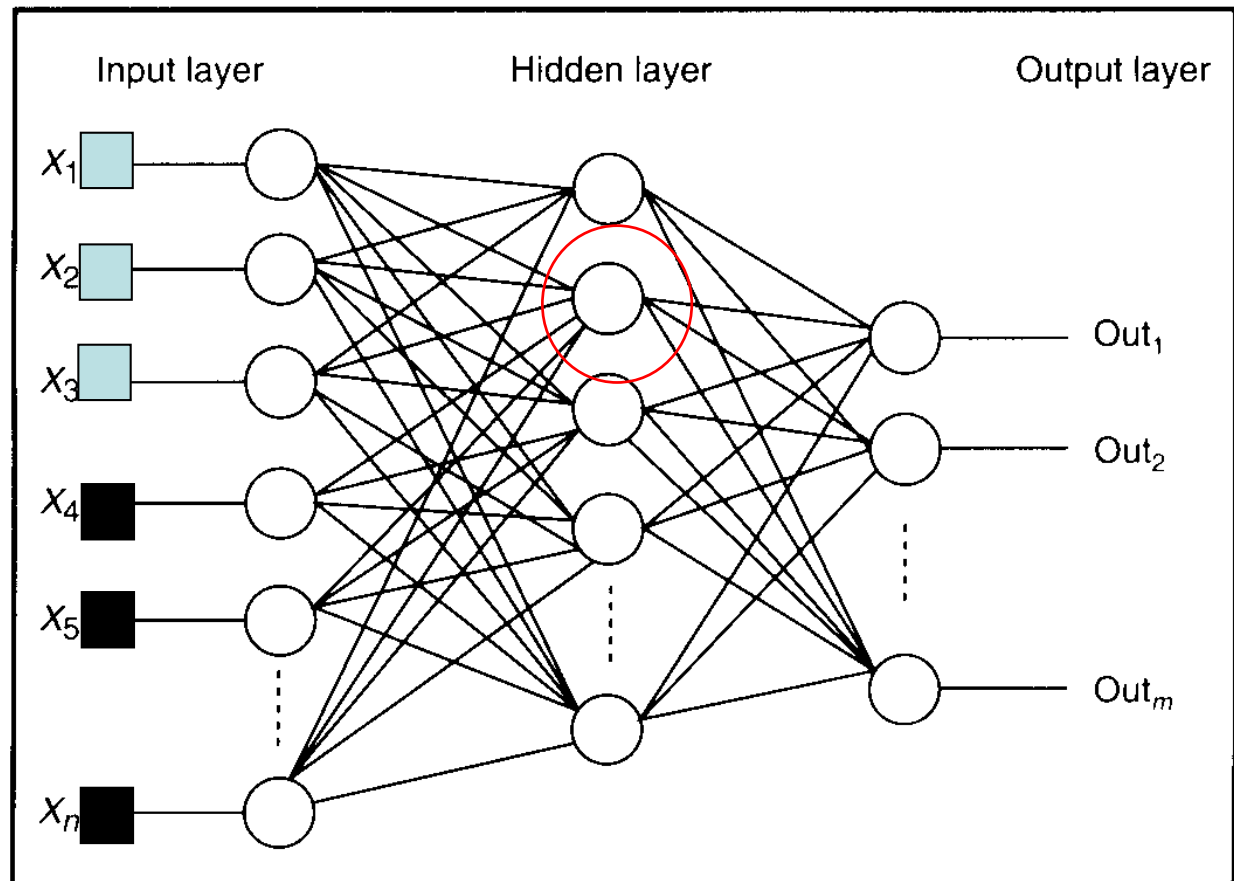
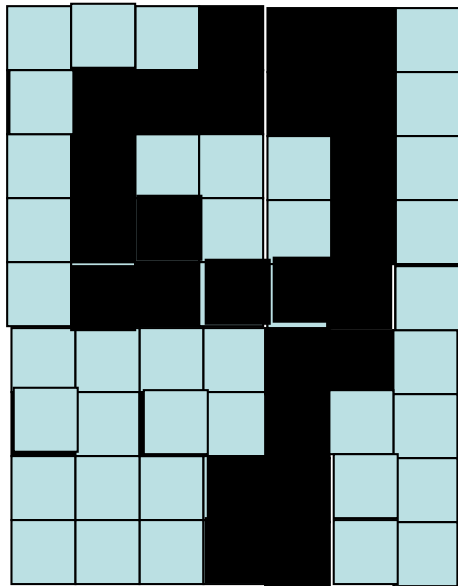


what's new is: algorithms for training many-layer networks

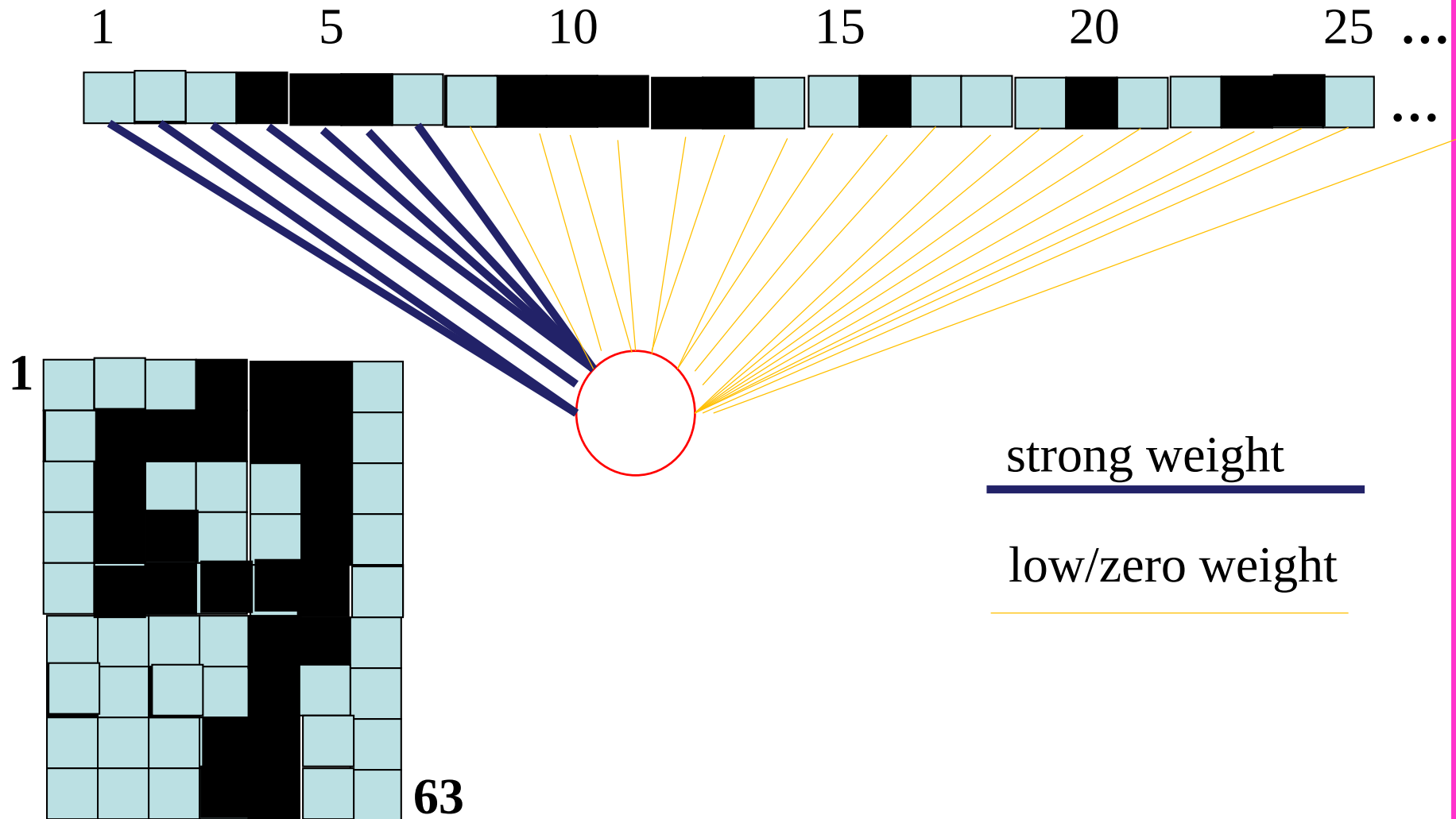
Feature detectors



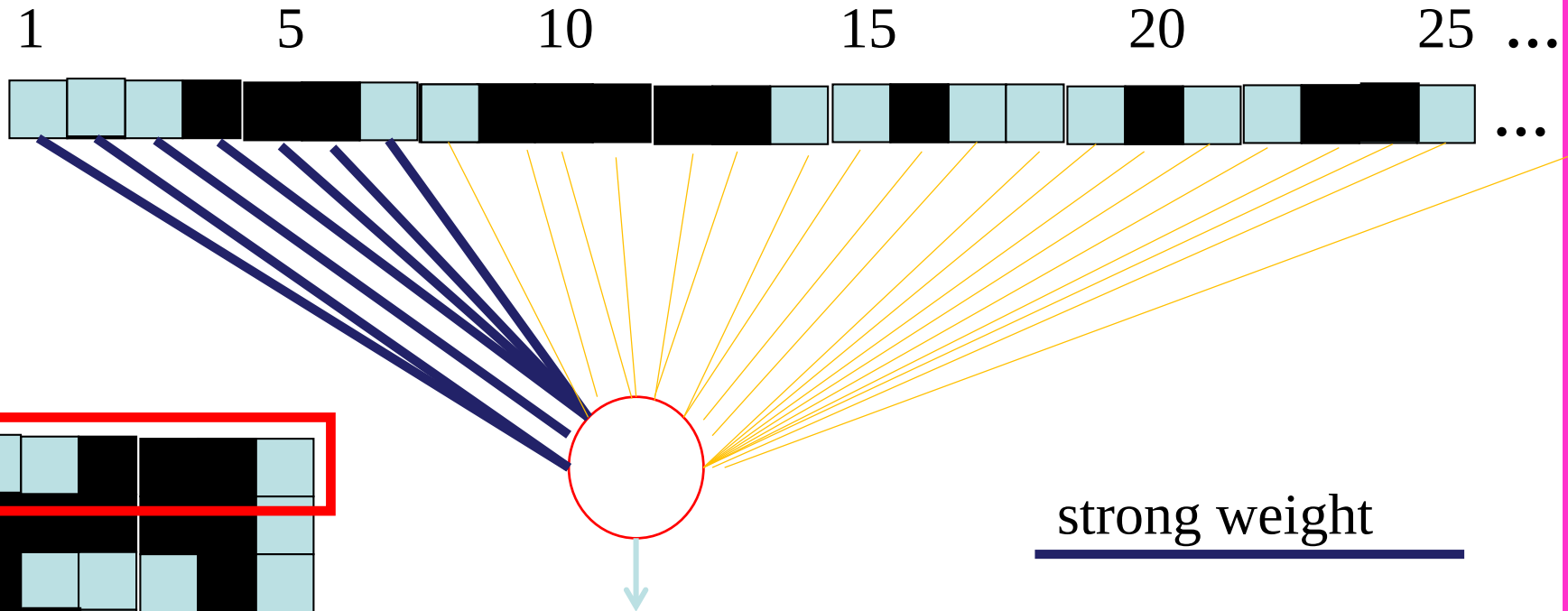
Feature detectors



Hidden layer units become *self-organised feature detectors*



What does this unit detect?

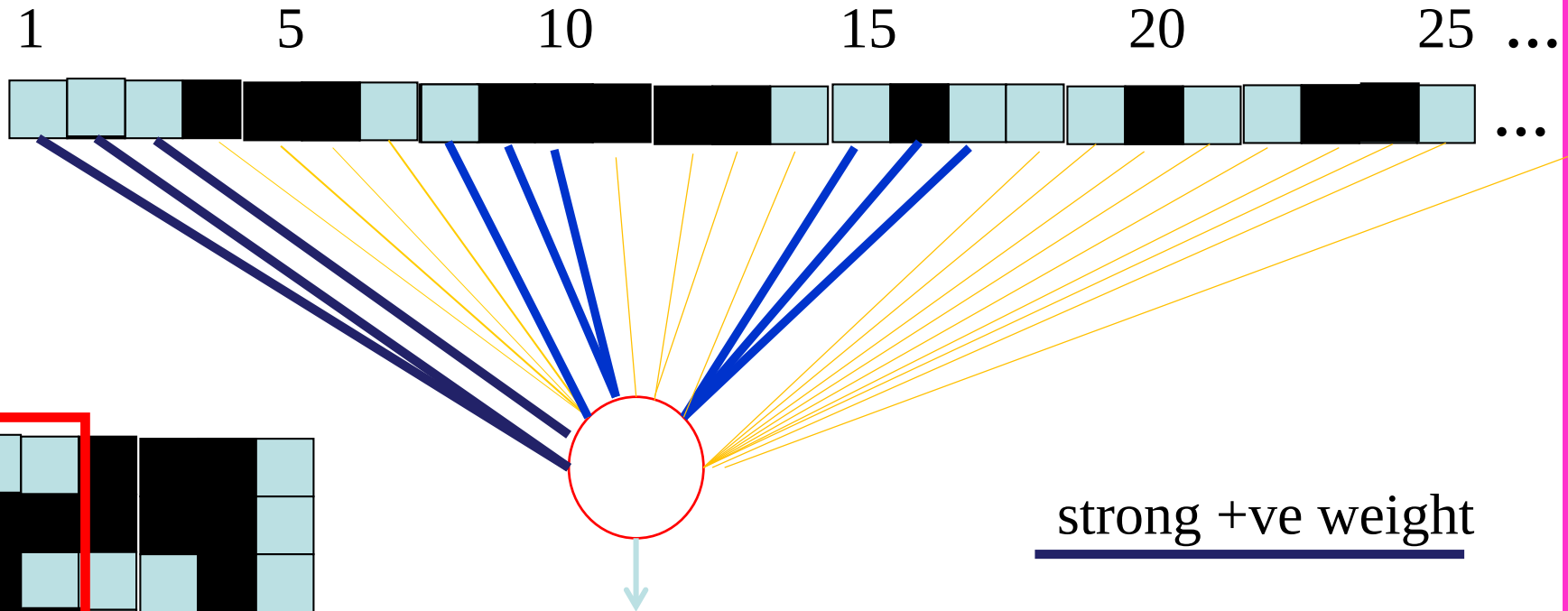


strong weight

low/zero weight

it will send strong signal for a horizontal line in the top row, ignoring everywhere else

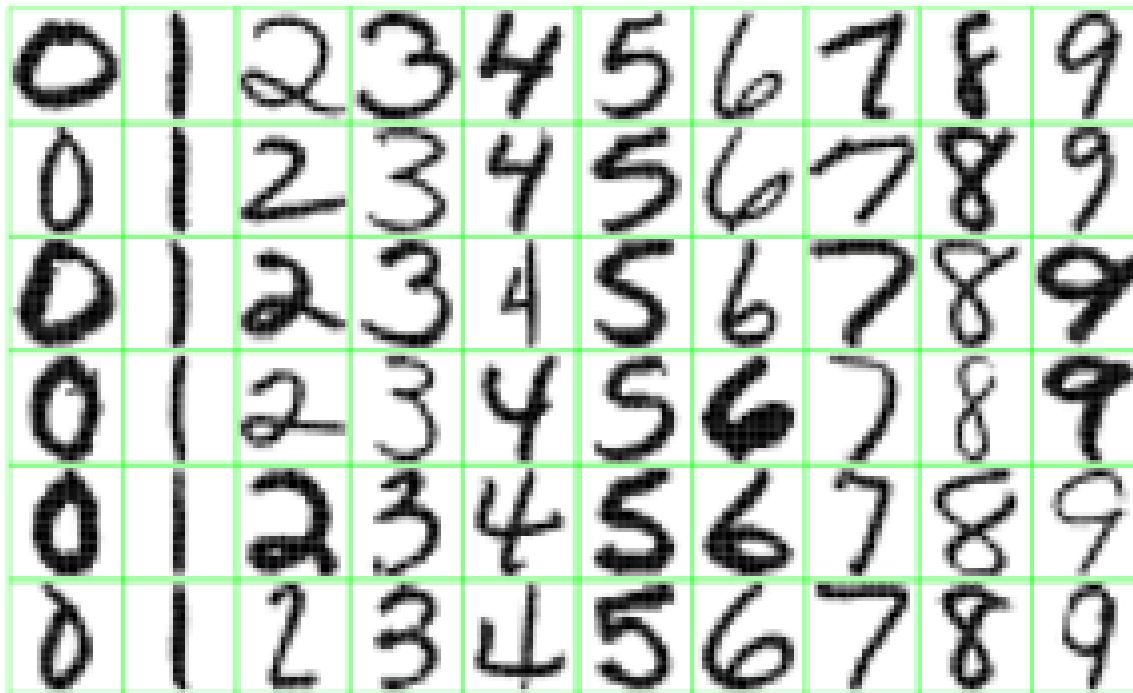
What does this unit detect?



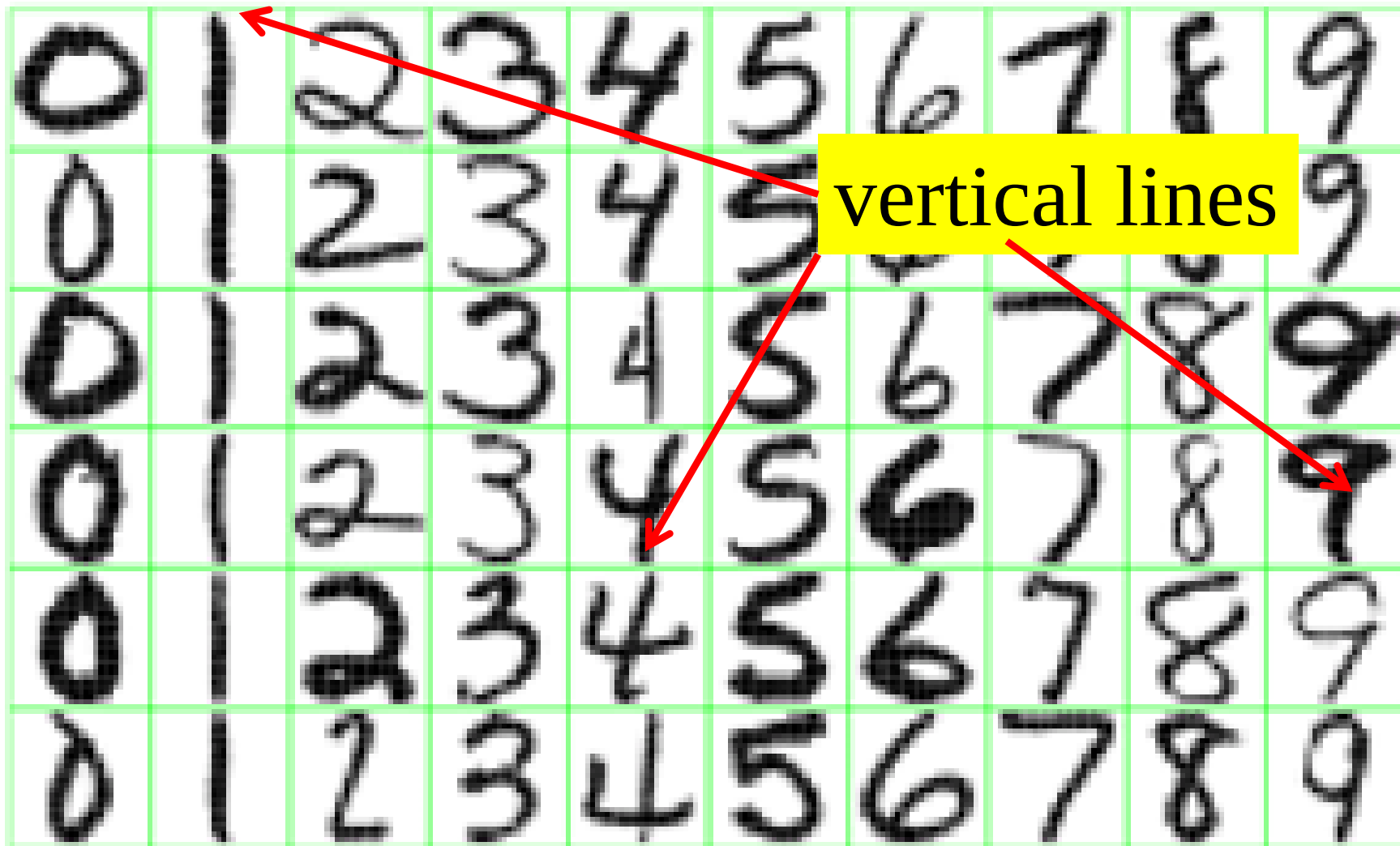
strong +ve weight

low/zero weight

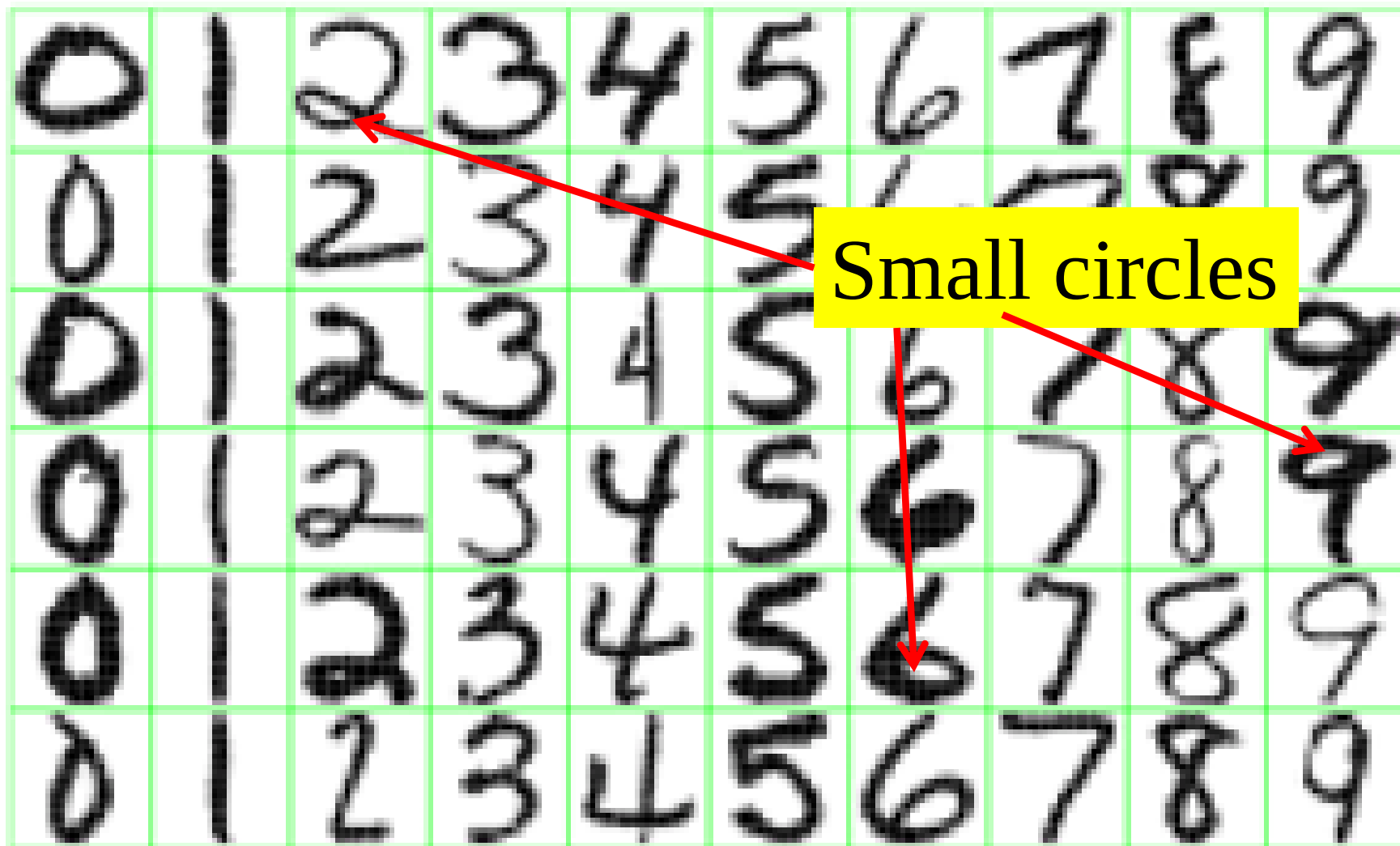
Strong signal for a dark area in the top left corner



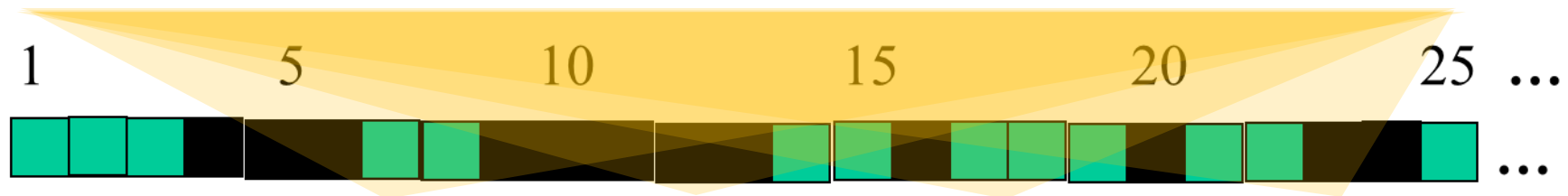
What features might you expect a good NN to learn, when trained with data like this?



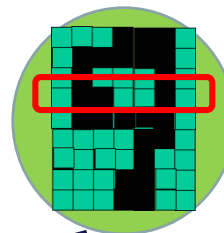
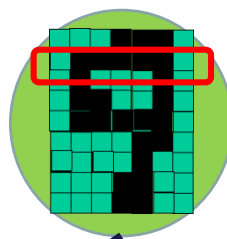
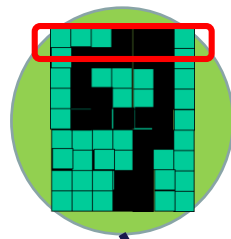




successive layers can learn higher-level features ...

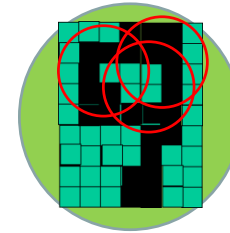
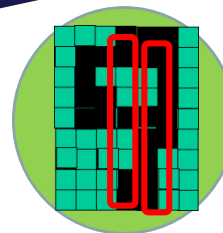
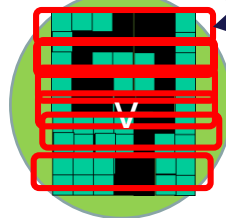


detect lines in
Specific positions



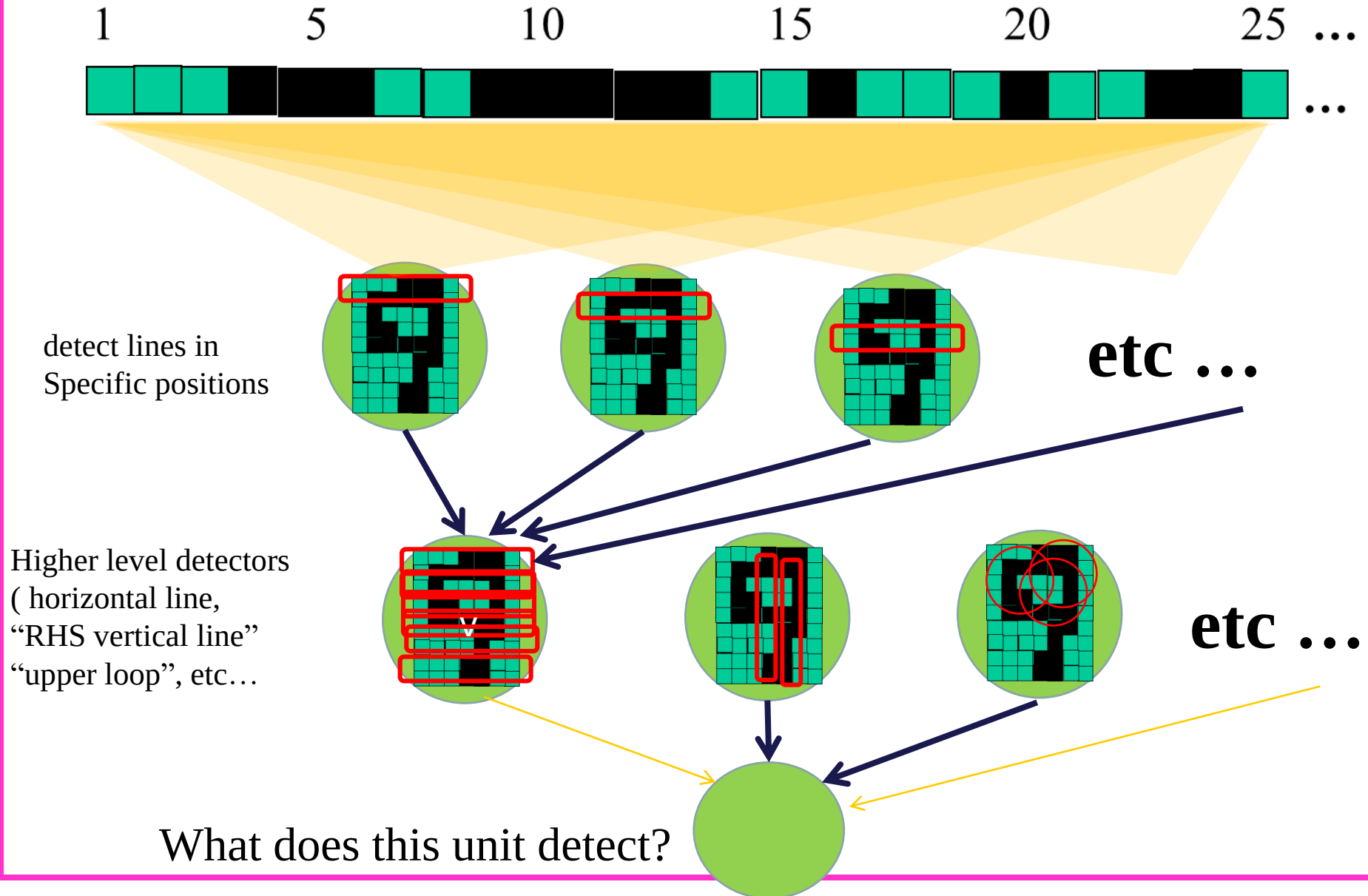
etc ...

Higher level detectors
(horizontal line,
“RHS vertical line”
“upper loop”, etc...

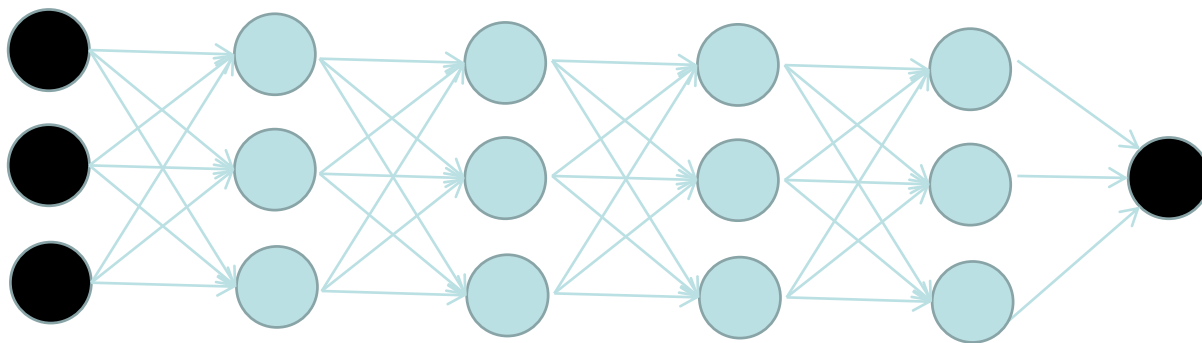


etc ...

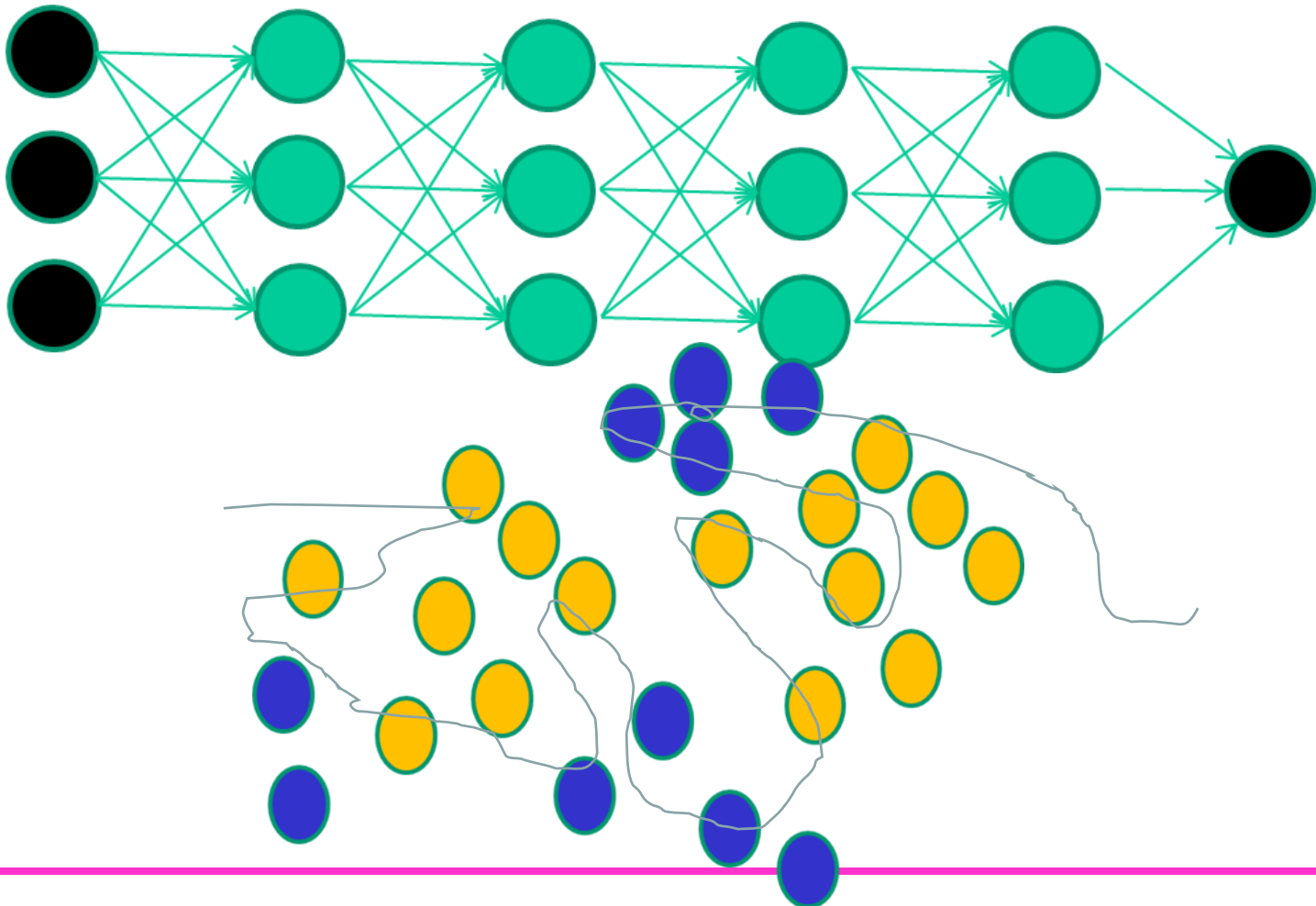
successive layers can learn higher-level features ...



*So: multiple layers make
sense*



But, until very recently, our weight-learning algorithms simply did not work on multi-layer architectures



CNN

Convolutional Neural Networks

- **Why?** For some types of data, especially for images MLP networks are not well adapted :
 - they are defined for **vectors** as input data,
 - images should be transformed into vectors,
 - the spatial information ?? Lost
 - learning was based on the extraction of features
 - these methods need a lot of experience for image processing.

1- CNN act directly on matrices, or even on tensors for images with three RGB color channels

2- CNN removes the manual extraction of features.

Convolutional Neural Networks

CNNs

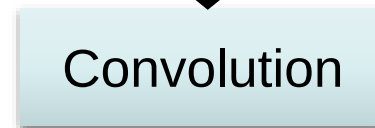
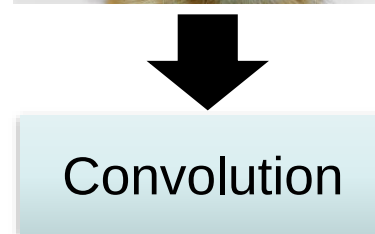
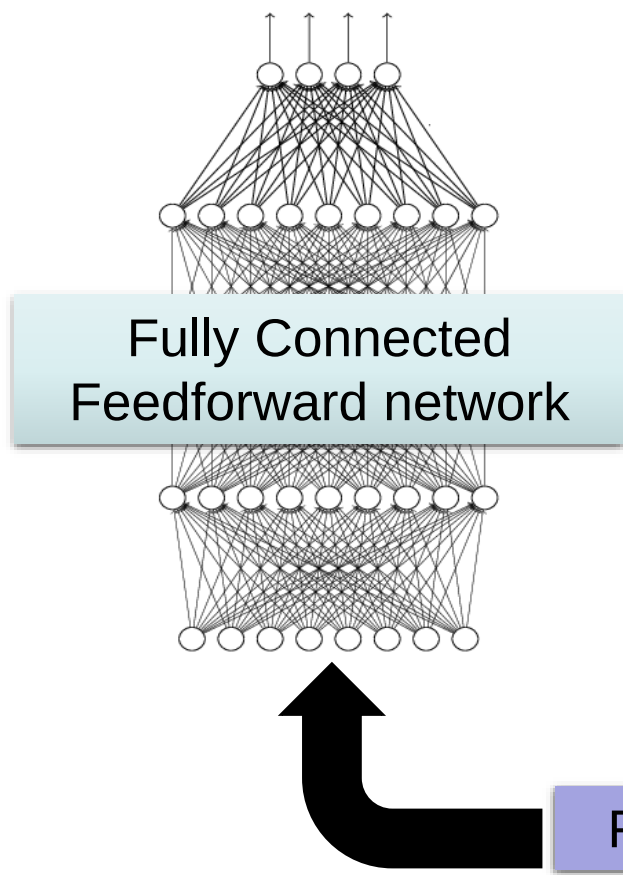
CNN

Convolutional Neural Networks

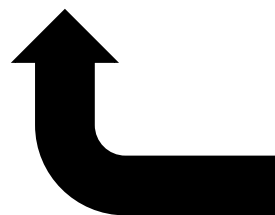
- Two main layer type:
 1. Convolution
 2. Max-pooling

The whole CNN

cat dog

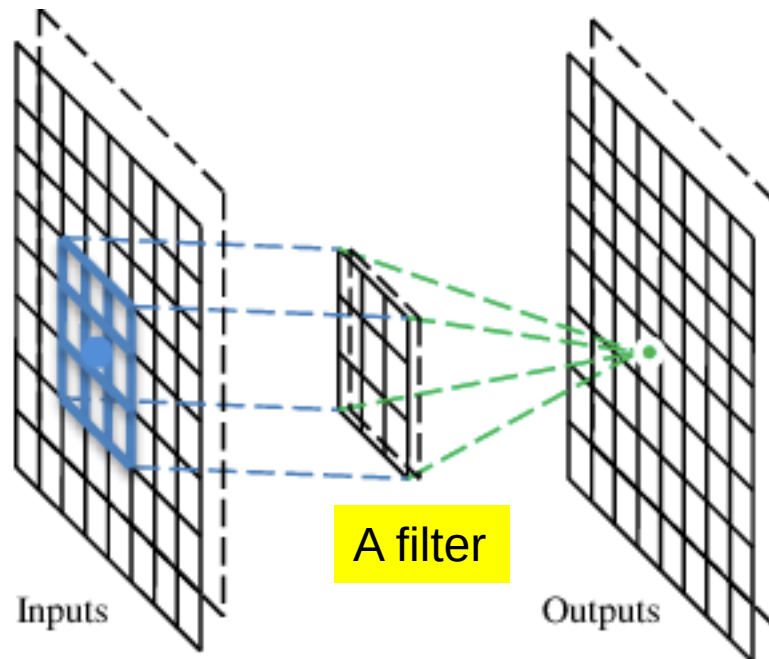


Can
repeat
many
times



A convolutional layer

A convolutional layer has a number of filters that does convolutional operation.



Convolution

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

These are the network parameters to be learned.

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

⋮ ⋮

Each filter detects a small pattern (3 x 3).

Convolution

stride=1

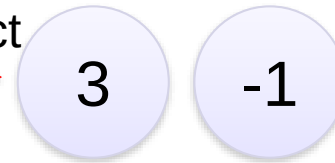
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

Dot
product



Convolution

If stride=2

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

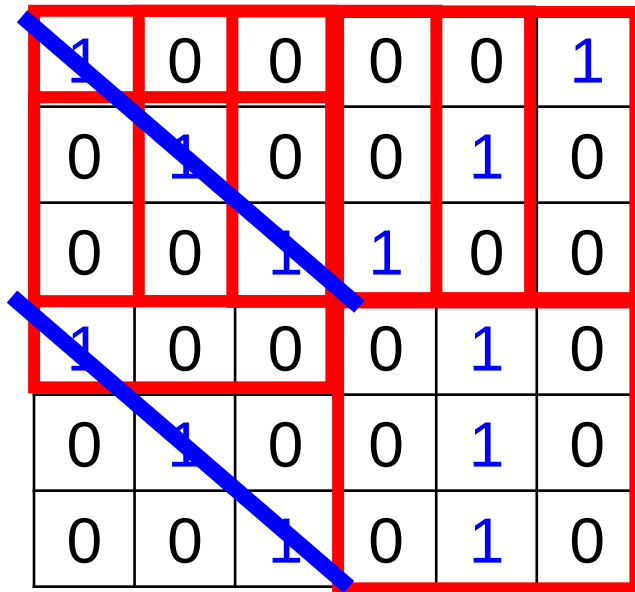
Filter 1

3

-3

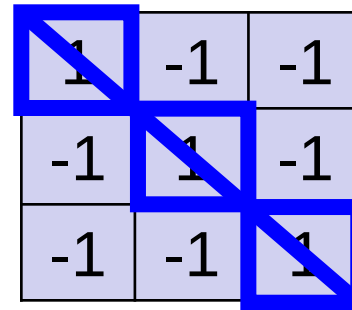
Convolution

stride=1



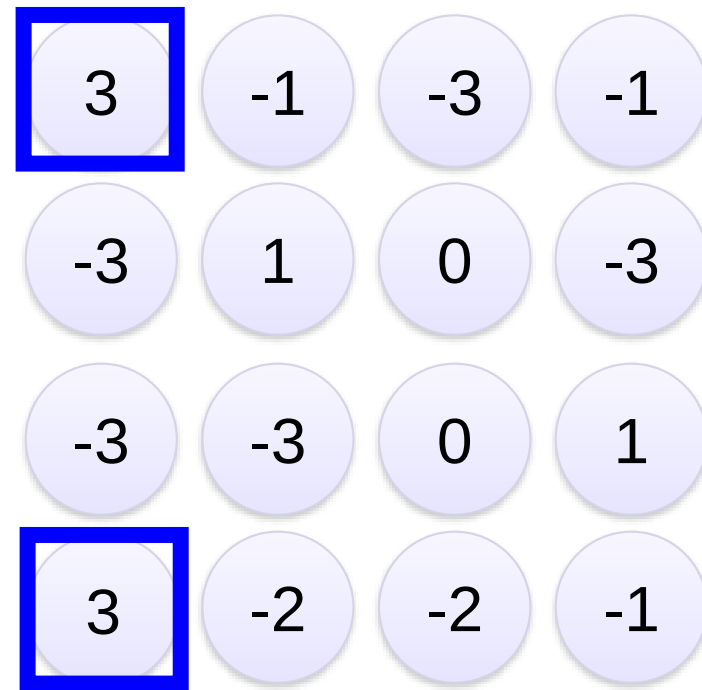
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image



1	-1	-1
-1	1	-1
-1	-1	1

Filter 1



3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

Convolution

stride=1

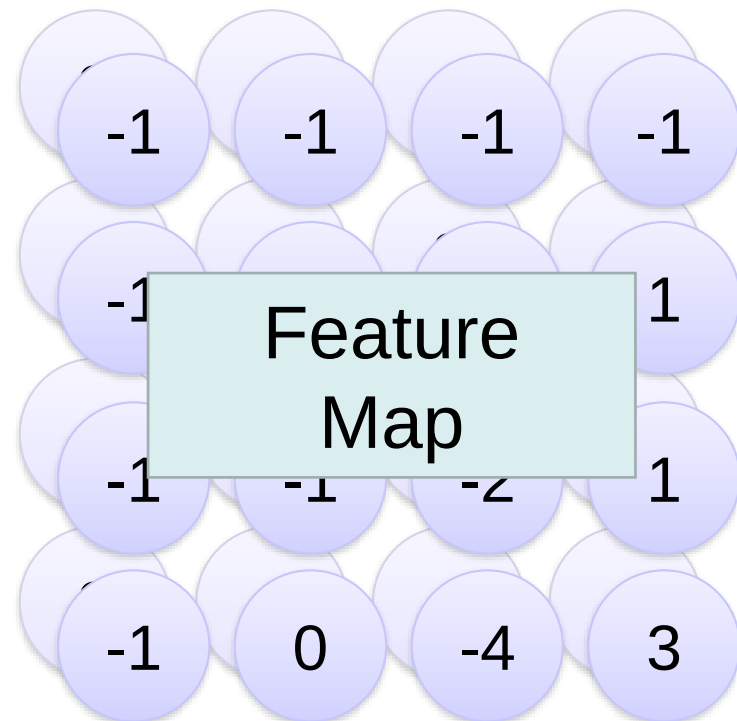
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

-1	1	-1
-1	1	-1
-1	1	-1

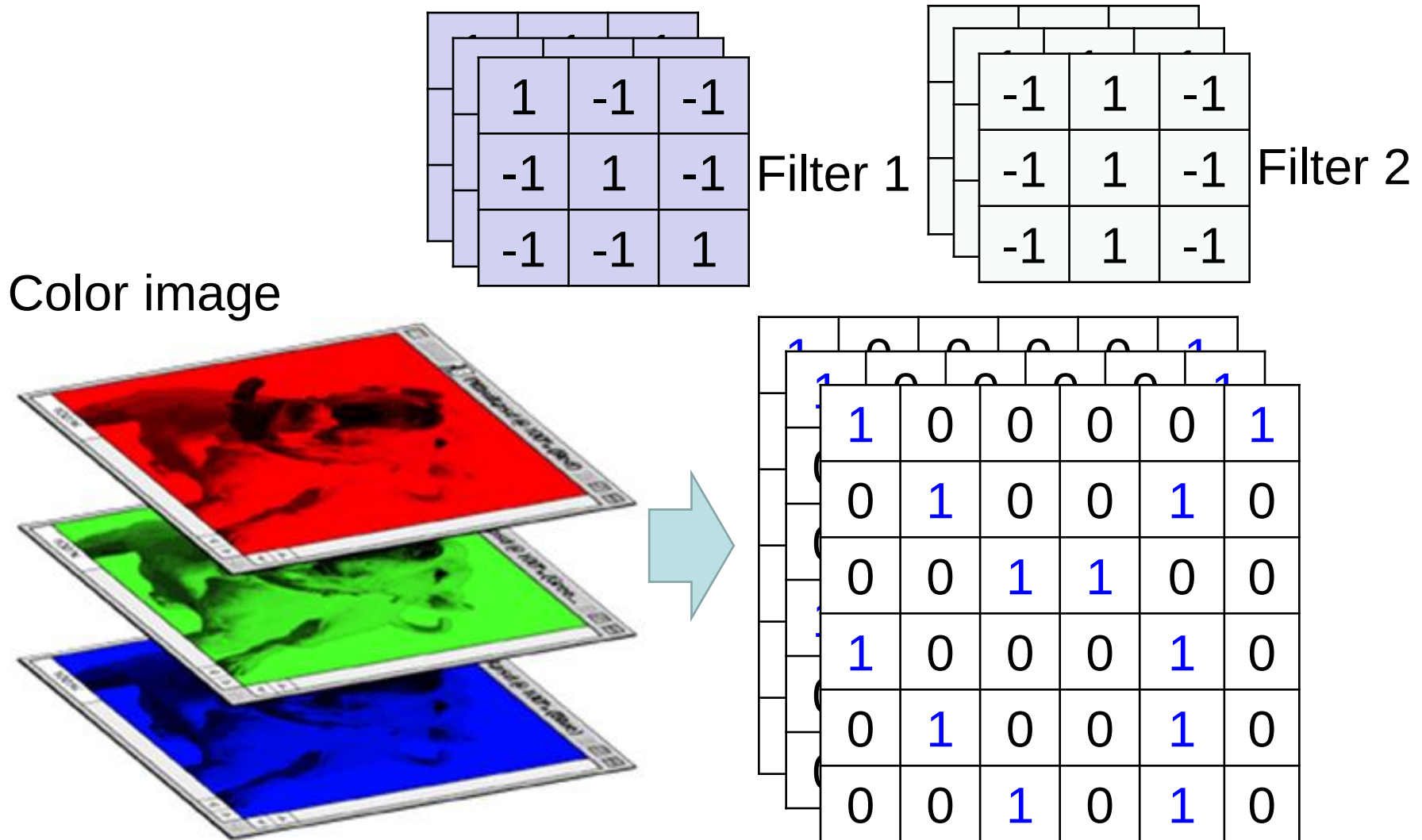
Filter 2

Repeat this for each filter



Two 4 x 4 images
Forming 2 x 4 x 4 matrix

Color image: RGB 3 channels



Convolution v.s. Fully Connected

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

image

1	-1	-1
-1	1	-1
-1	-1	1

-1	1	-1
-1	1	-1
-1	1	-1

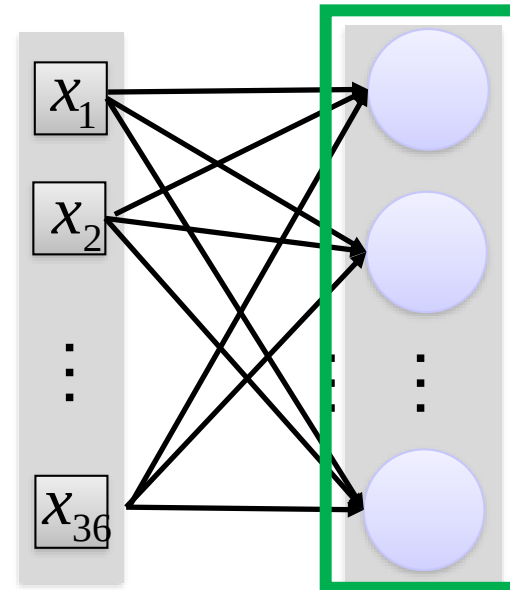


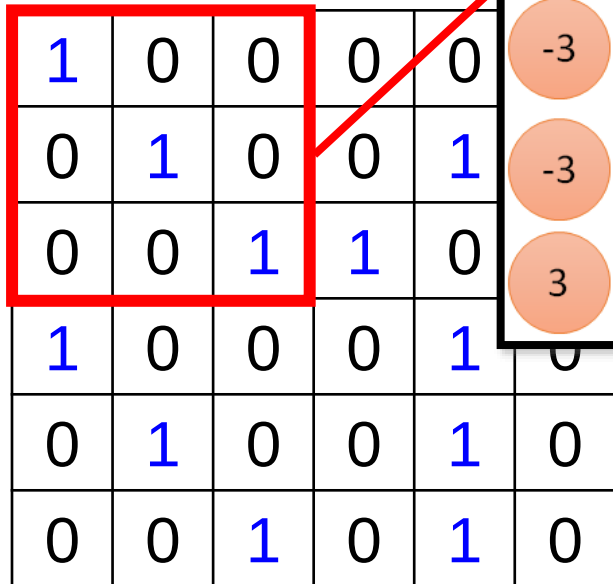
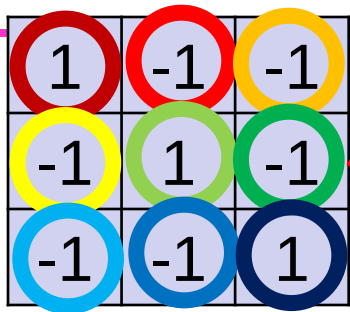
convolution

-1	-1	-1	-1
-1	-1	-2	1
-1	-1	-2	1
-1	0	-4	3

Fully-
connected

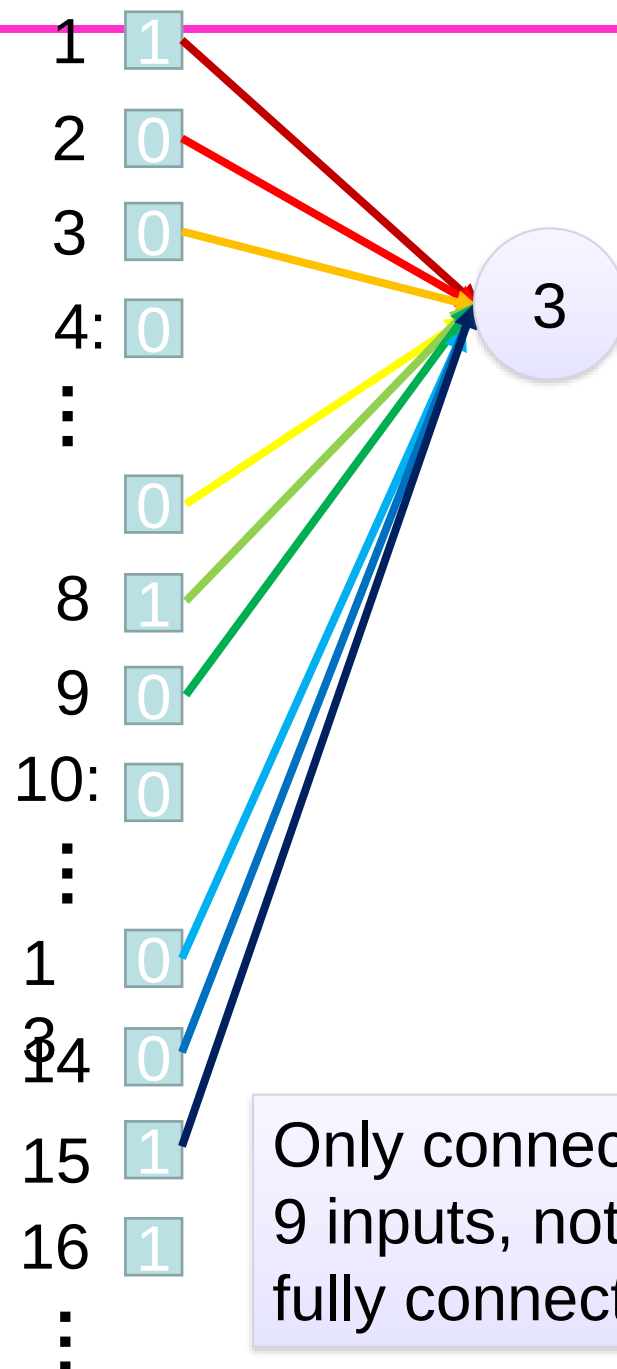
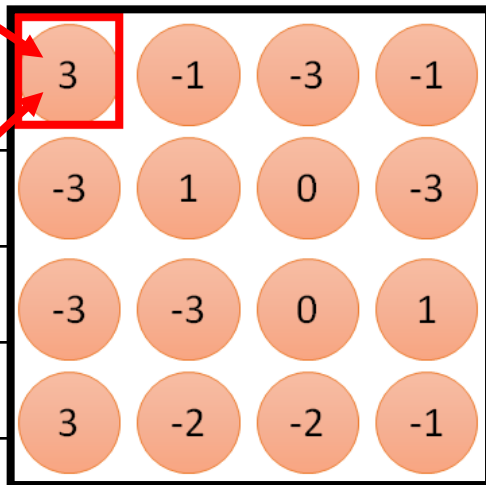
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

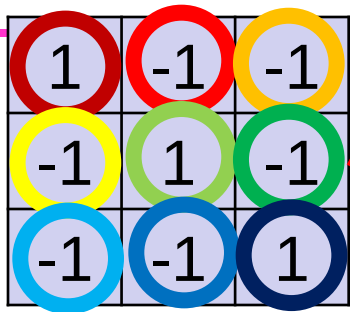




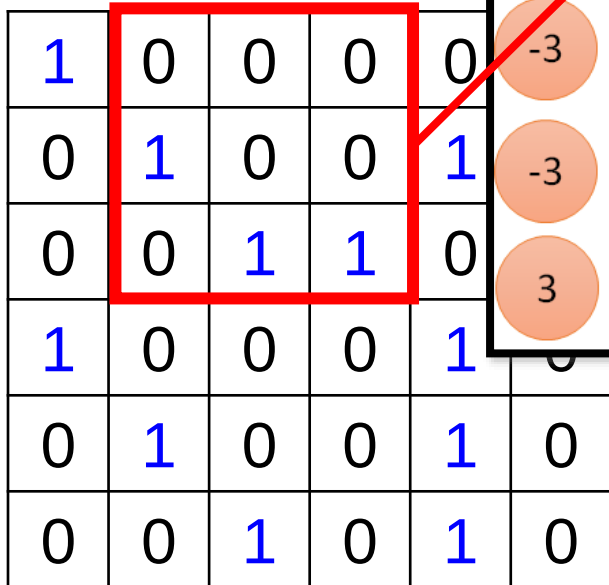
6 x 6 image

fewer parameters!





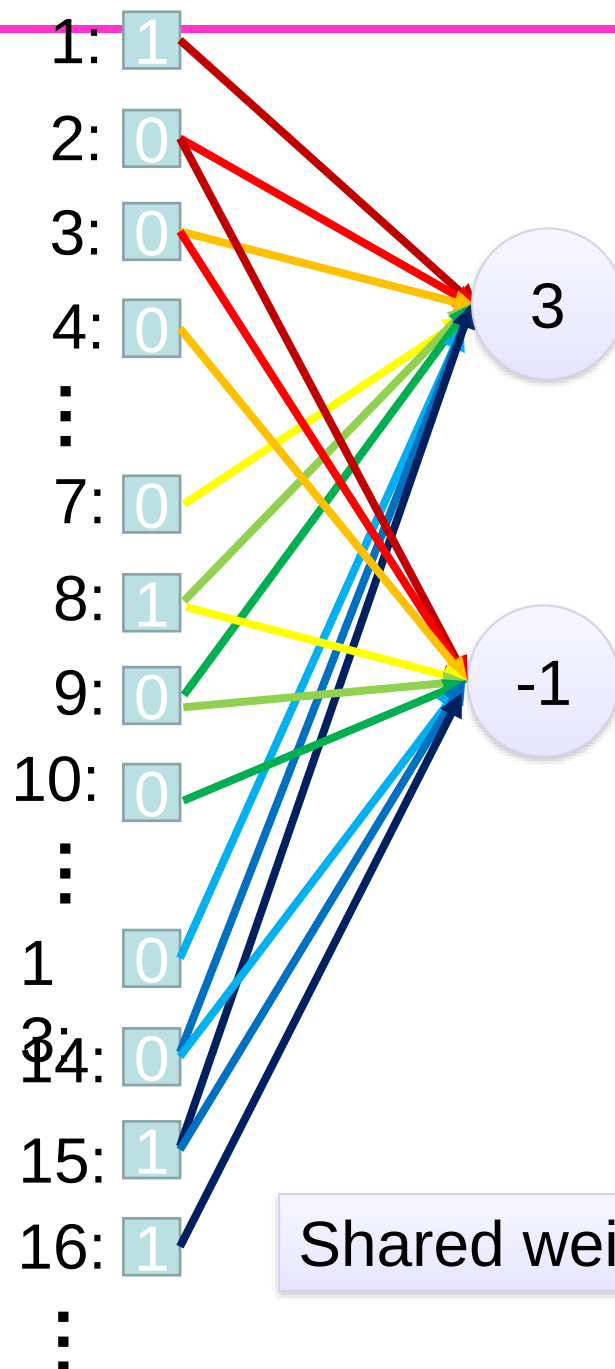
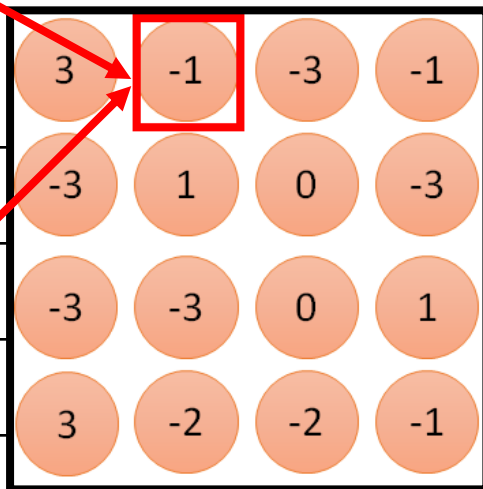
Filter 1



6 x 6 image

Fewer parameters

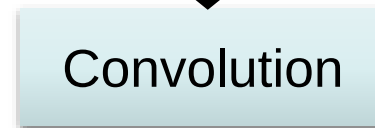
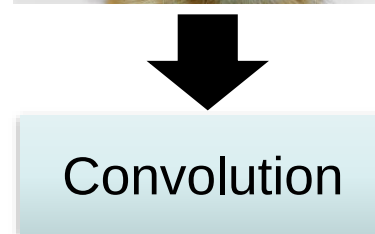
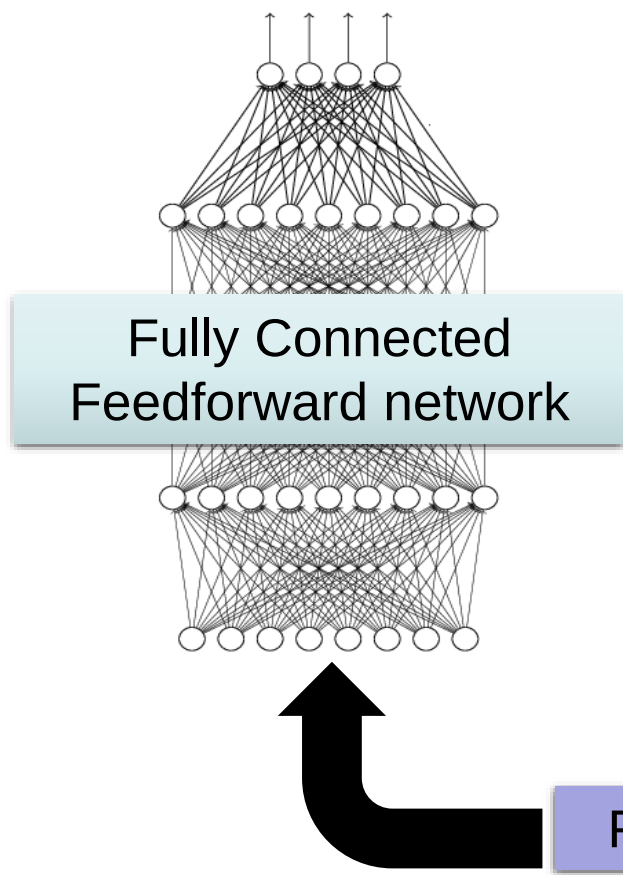
Even fewer parameters



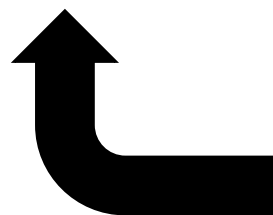
Shared weights

The whole CNN

cat dog



Can
repeat
many
times



Max Pooling

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

-1	-1	-1	-1
-1	-1	-2	1
-1	-1	-2	1
-1	0	-4	3

Why Pooling

- Subsampling pixels will not change the object

bird



Subsampling

bird



We can subsample the pixels to make image smaller

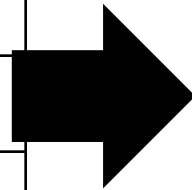


fewer parameters to characterize the image

Max Pooling

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

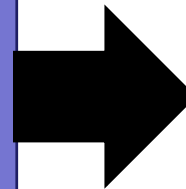
6 x 6 image



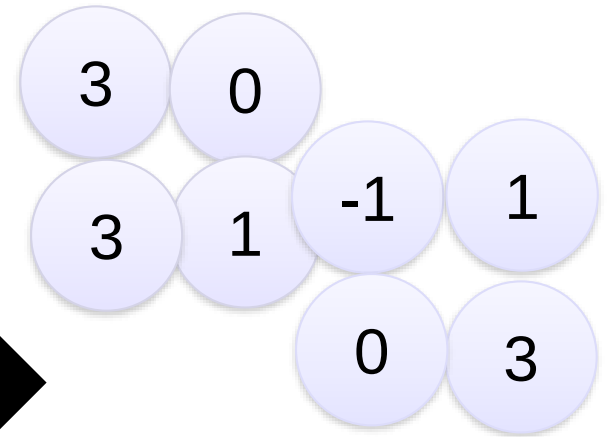
Conv



Max
Pooling



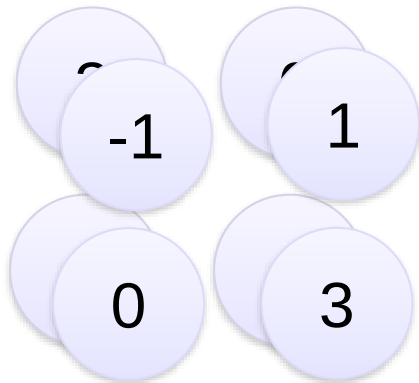
New image
but smaller



2 x 2 image

Each filter
is a channel

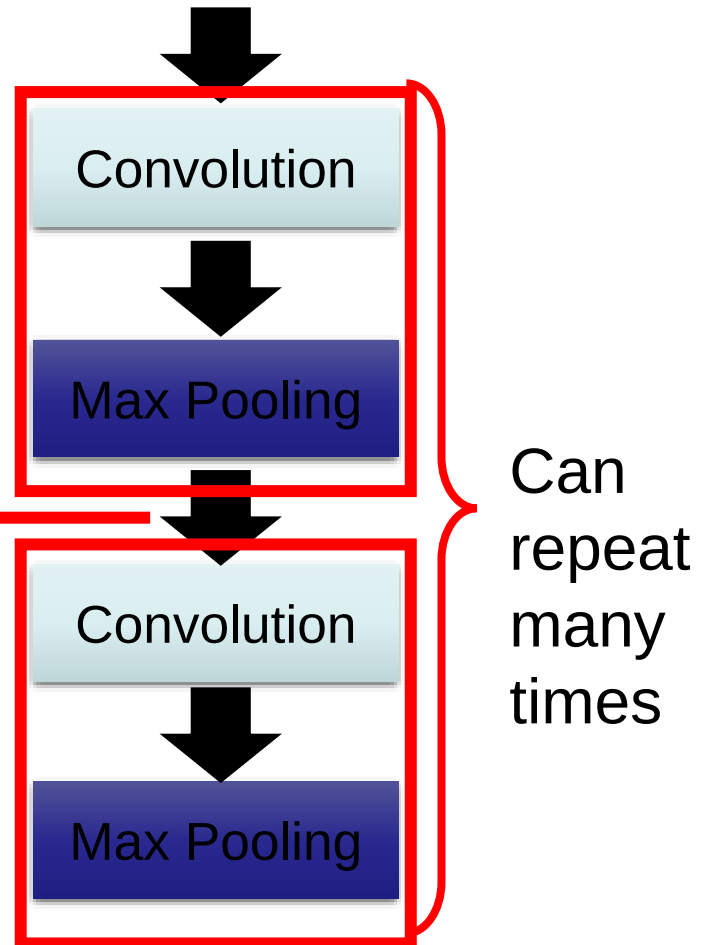
The whole



A new image

Smaller than the original image

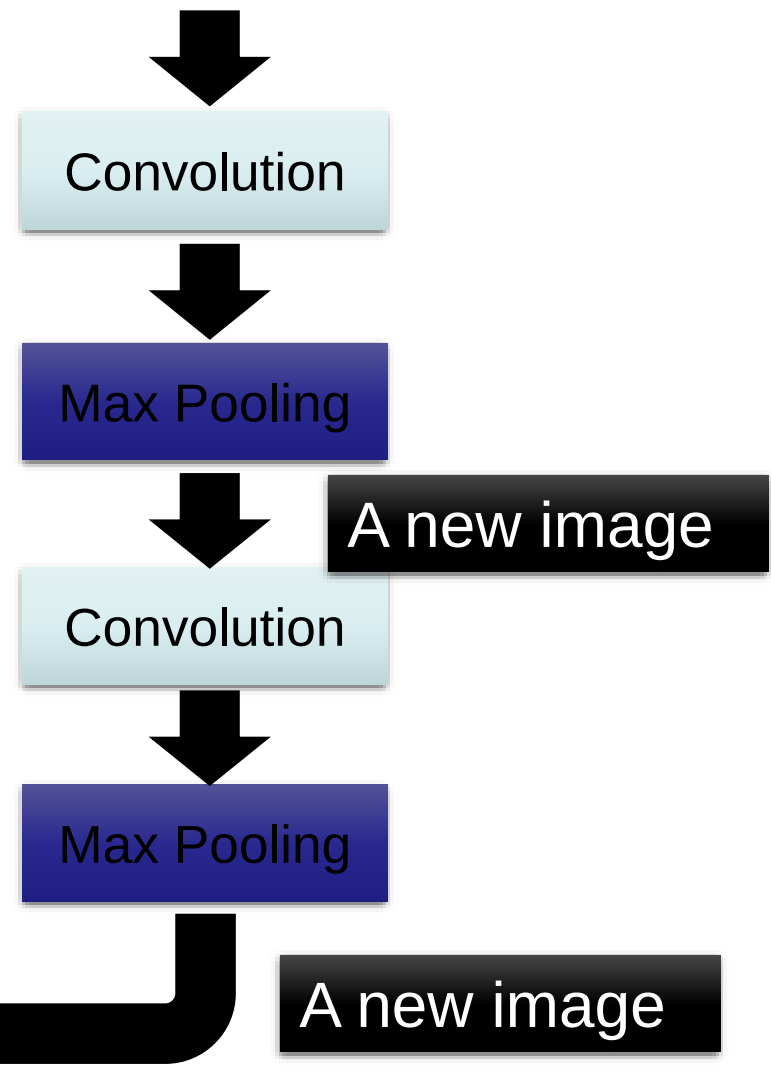
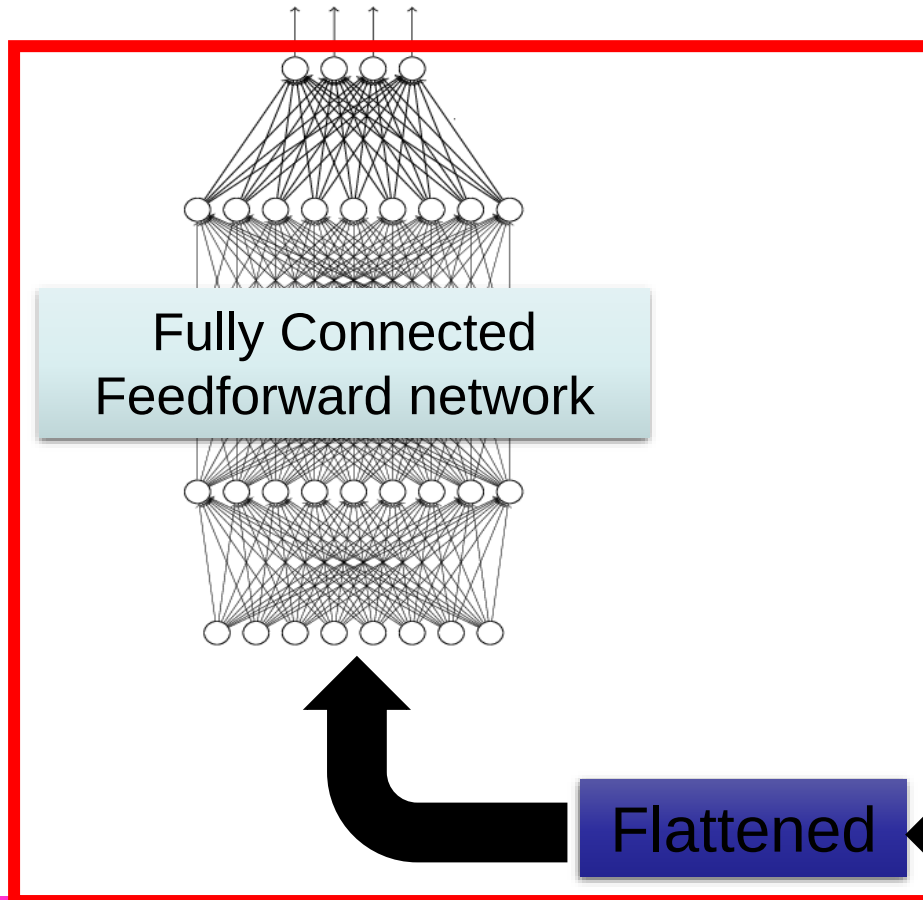
The number of channels is the number of filters



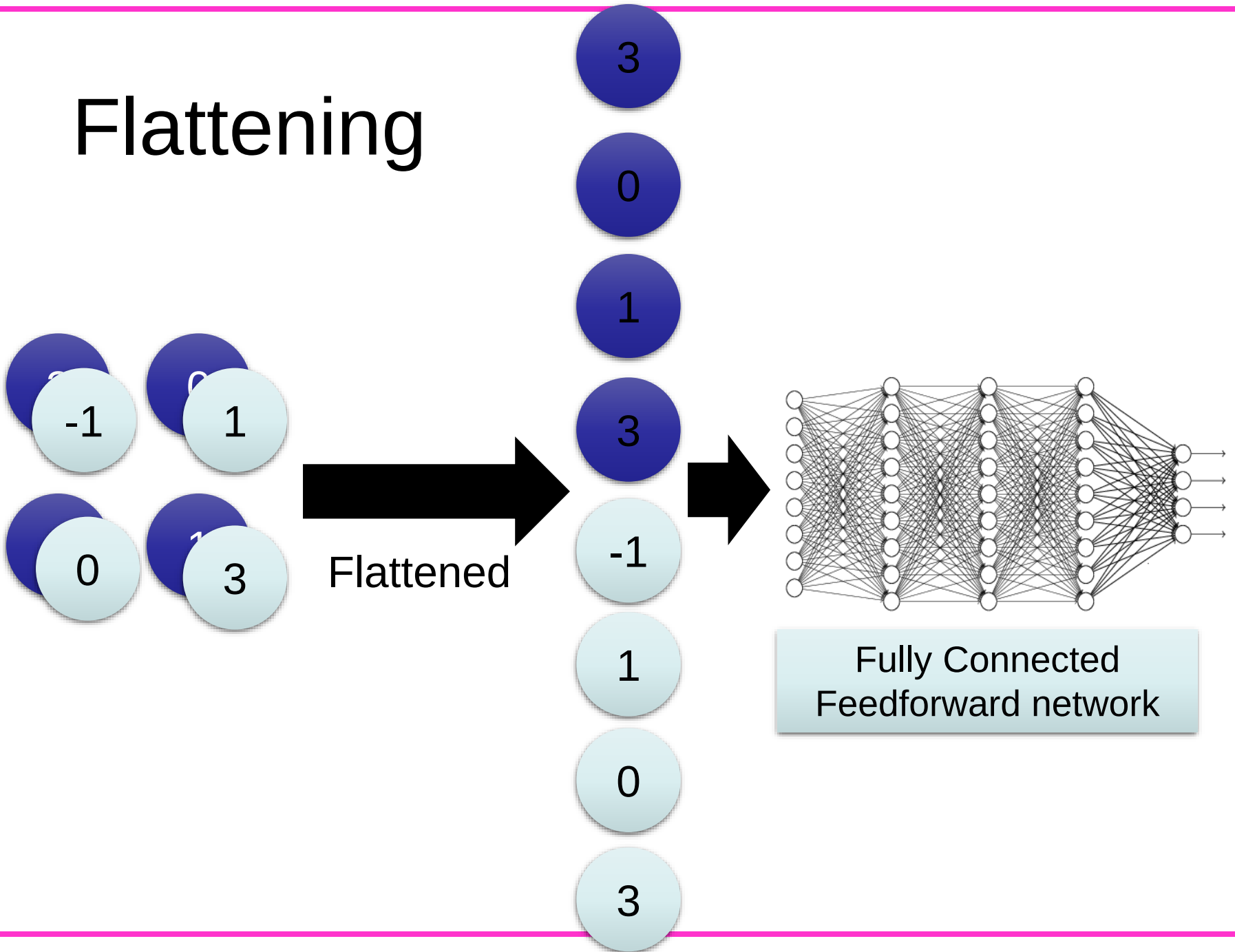
The whole



cat dog



Flattening



CNN in Keras

Only modified the *network structure* and *input format (vector -> 3-D tensor)*

```
model2.add( Convolution2D( 25, 3, 3,  
                           input_shape=(28, 28, 1)) )
```

1	-1	1
-1	1	-1
-1	-1	-1

-1	1	-1
-1	1	-1
-1	1	-1

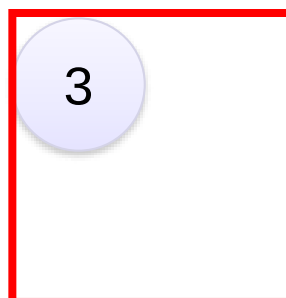
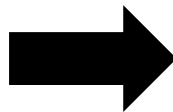
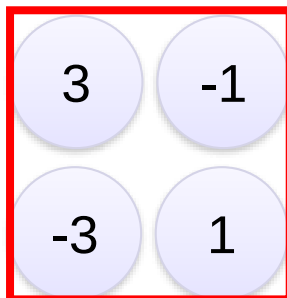
There are
25 3x3
filters.

Input_shape = (28 , 28 , 1)

28 x 28 pixels

1: black/white, 3: RGB

```
model2.add(MaxPooling2D( (2, 2) ))
```



input

Convolution

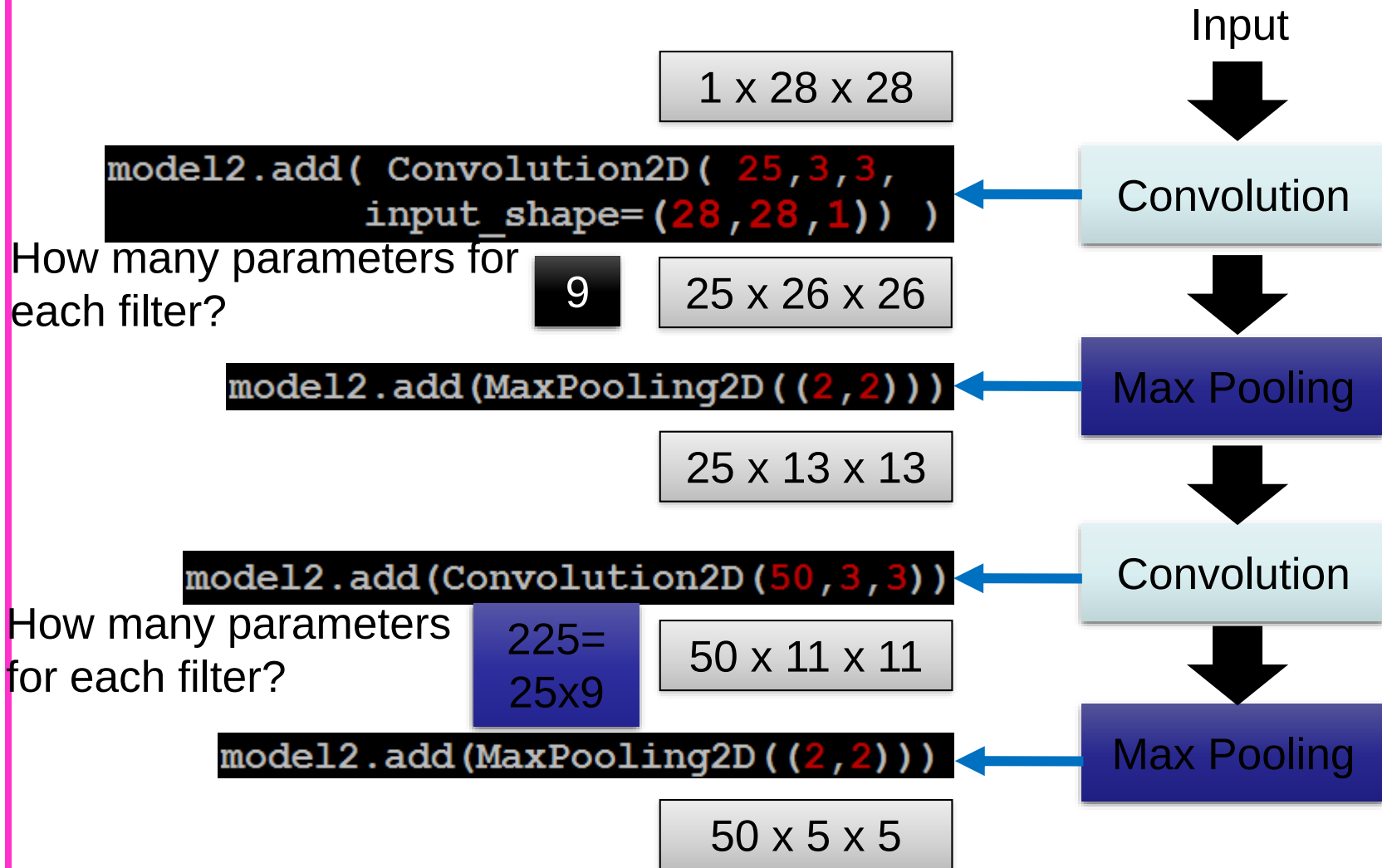
Max Pooling

Convolution

Max Pooling

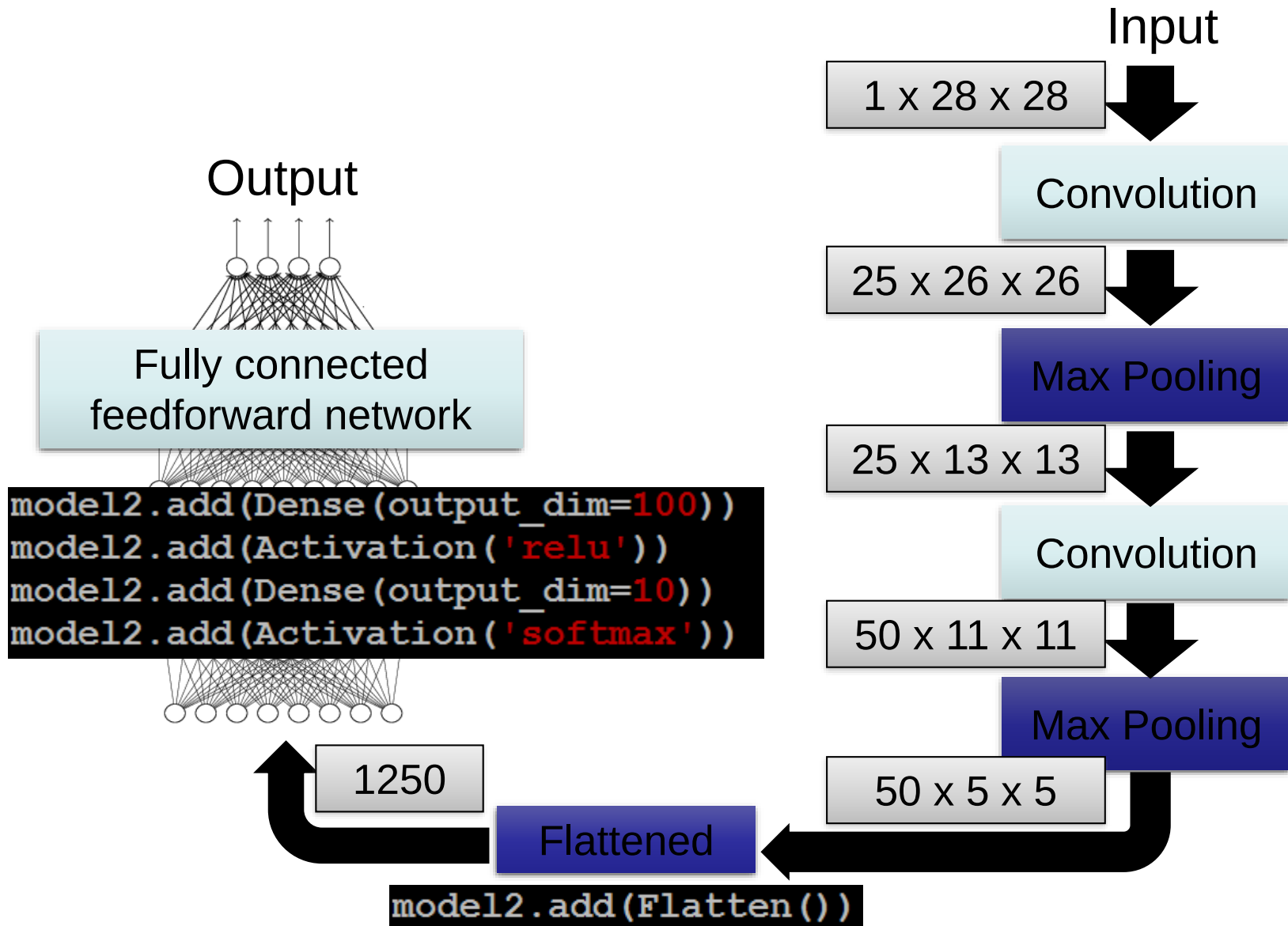
CNN in Keras

Only modified the *network structure* and *input format (vector -> 3-D array)*

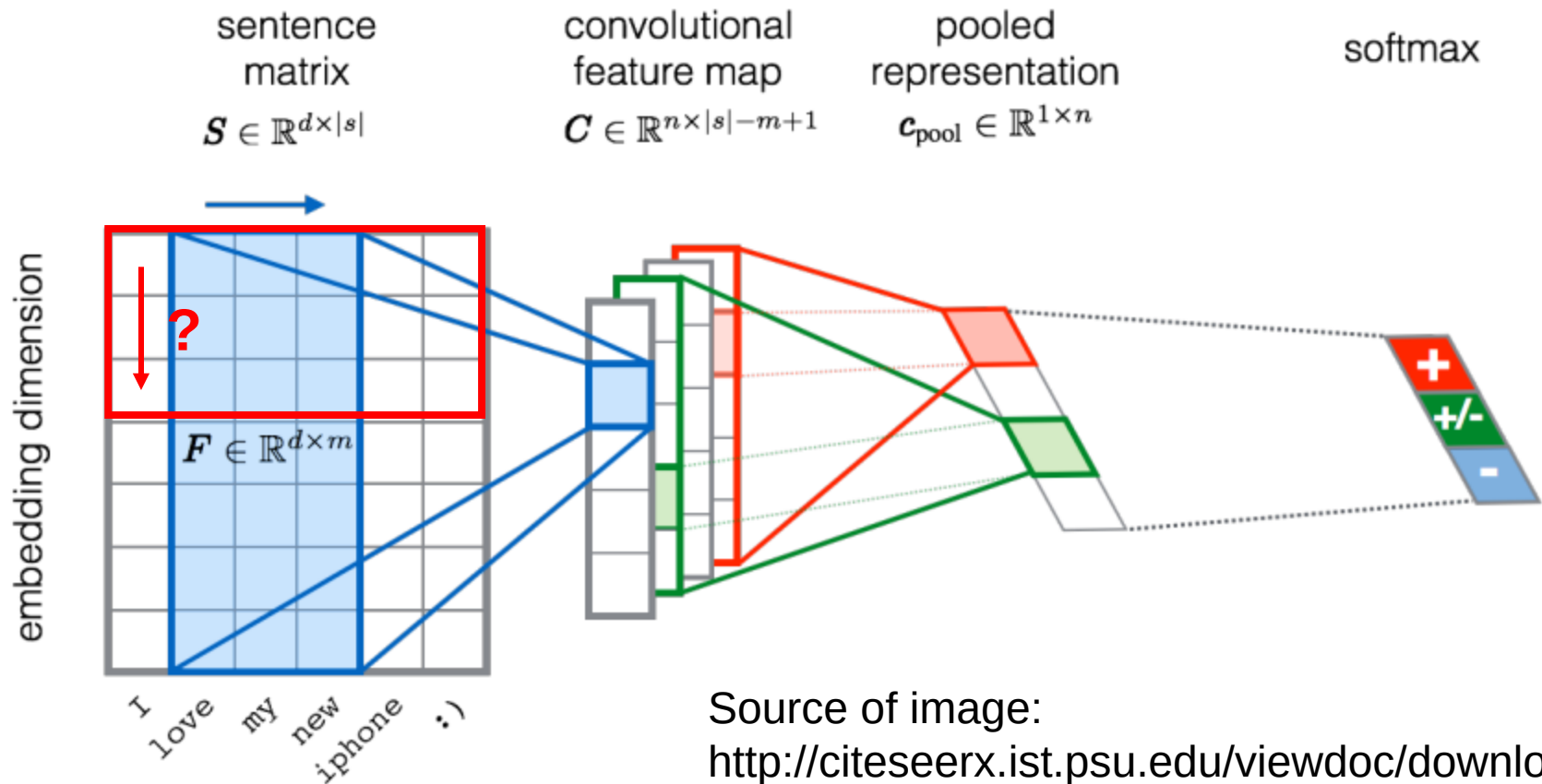


CNN in Keras

Only modified the *network structure* and *input format (vector -> 3-D array)*



CNN in text classification



Source of image:

<http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.703.6858&rep=rep1&type=pdf>

A CNN compresses a fully connected network

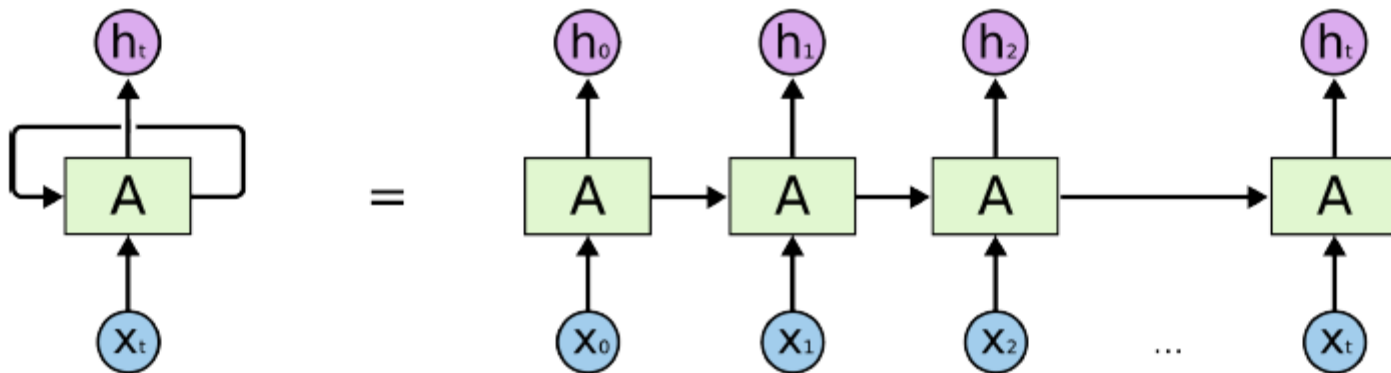
- Reducing number of connections
- Shared weights on the edges
- Max pooling further reduces the complexity

Long Short Term Memory Networks

(LSTMs)

Recurrent neural networks (1980)

- Why ?
 - In order to infer sequential data such as text or time series



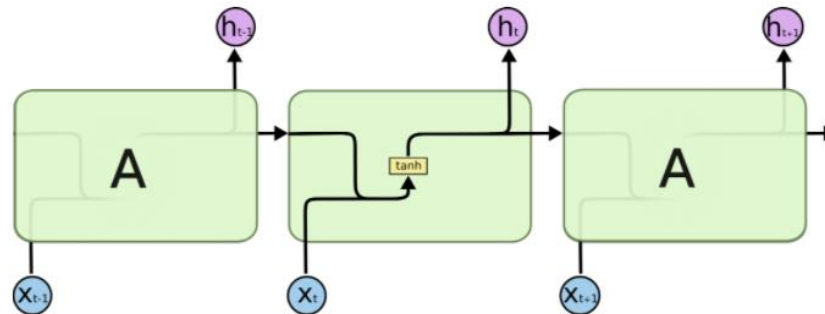
- Challenge:
 - to learn long time dependencies.

RNN

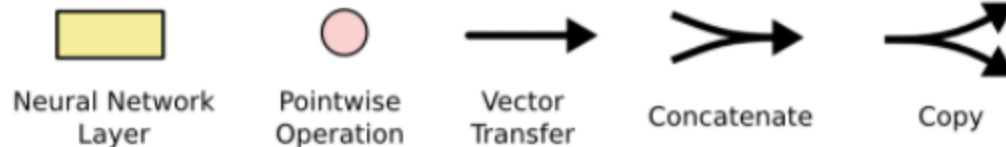
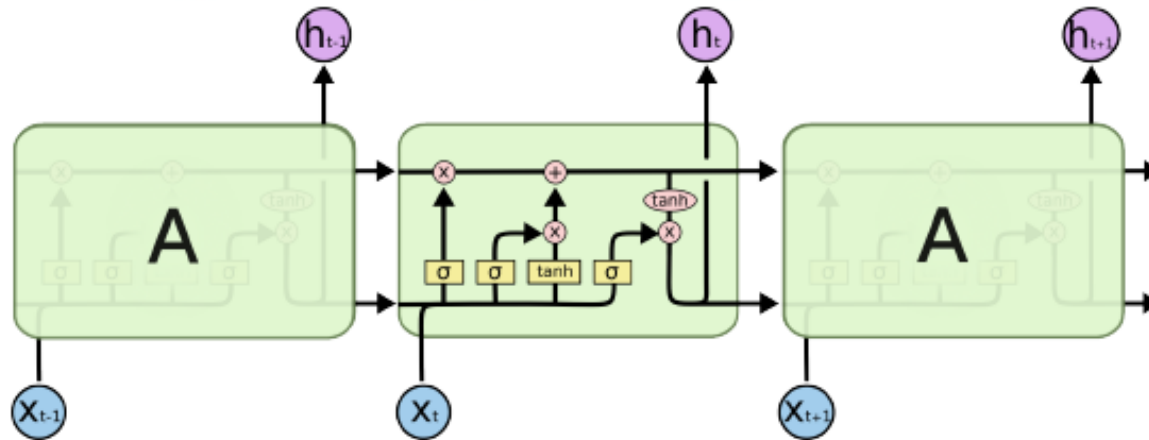
- I grew up in France... I speak fluent *French*.
- **Predict the last word**
 - Recent information → It is probably the name of a language
 - which language → we need the context of France
 - Unfortunately, as that gap grows, RNNs become unable to learn to connect the information.

Long Short Term Memory Networks

RNN:

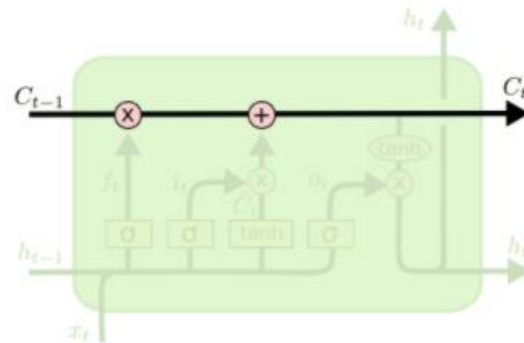


LSTM:



LSTM

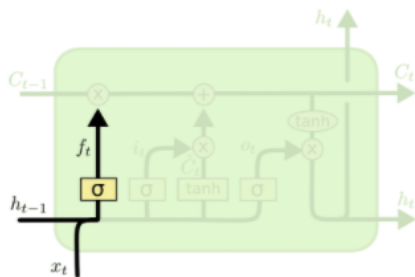
- The key to LSTMs is the **cell state**
- It has the ability to remove or add information to the cell state
- An LSTM has three gates, to protect and control the cell state.



forget gate layer

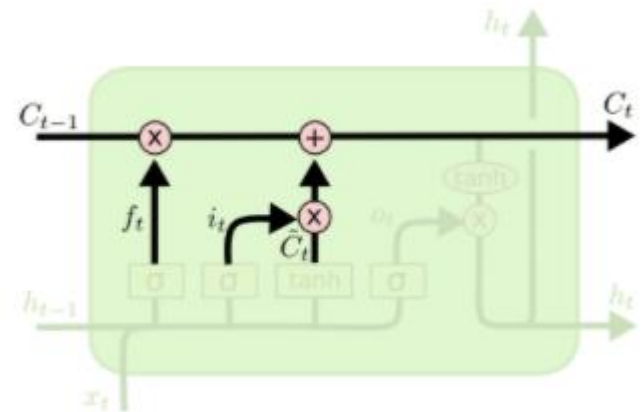
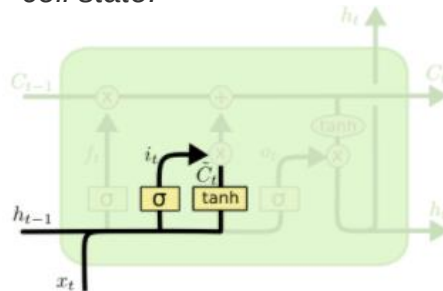
Outputs $f_t : 0..1$

1: completely keep this
0: completely get rid of this.



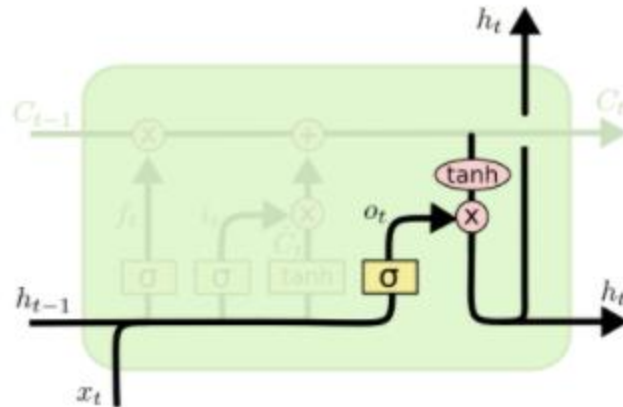
input gate layer

to decide what new information we're going to store in the cell state.

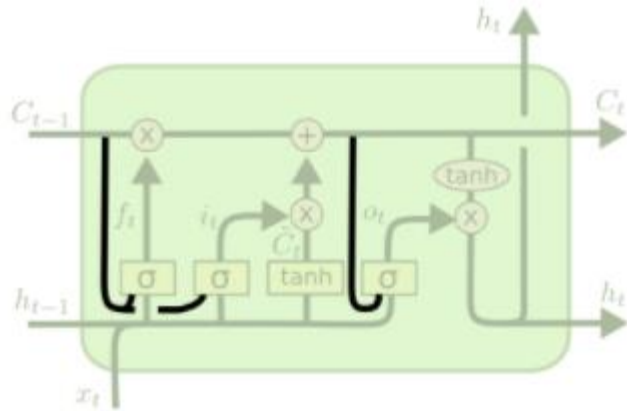


LSTM

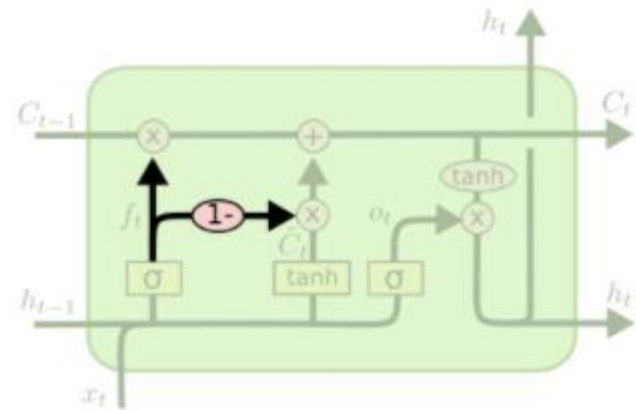
- Finally, we need to decide what we're going to output



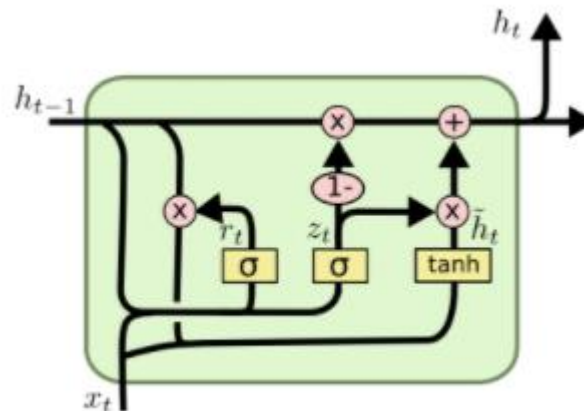
LSTM variations



(1) It lets the gate layers look at the cell state.



(2) It combines forget and input gates



(3) It combines forget and input gates
It also merges the cell state and hidden state

Autoencoders

Autoencoders

- An autoencoder has a structure very similar to a feedforward neural network
- **the primary difference**
 - unsupervised
 - the number of neurons in the output layer are equal to the number of inputs.
- Autoencoder uses hidden layers to try to re-create the inputs.
- It can be used for dimension reduction purposes $\text{minimize } L = f(X, X')$
- *stacked autoencoders* (or *deep autoencoders*): autoencoders with more than one layer

